Perception for Smart Bike Lights: Real-Time Embedded Bike Headlight Control Using a Single-Model Convolutional Neural Network

Sagnik Das
Technische Universität Nürnberg
sagnik.das@utn.de
ORCID: 0009-0009-4601-3660

Muhid Abid Siddiqui
Technische Universität Nürnberg
muhid.abid.siddiqui@utn.de
ORCID: 0009-0007-7194-7954

Nitish Devrani
Technische Universität Nürnberg
nitish.devrani@utn.de
ORCID: 0009-0000-0877-4383

***Abstract*—Adaptive bike headlights reduce rider workload and improve safety, but robust control requires a perception model that runs in real time on low-power hardware and generalizes across diverse streets and weather. We designed a vision-only pipeline that maps a forward-facing camera stream to discrete beam states (`off`/`dim`/`high`) in a safety-first policy that avoids glare while preserving rider visibility. Our final system combines illumination and vehicle-proximity cues into a single lightweight residual CNN, trained on BDD100K using derived labels from time-of-day attributes and hazard bounding boxes, and deployed on a Raspberry Pi that drives GPIO outputs for headlight control. On the BDD100K split derived from our policy-based labeling scheme, the model achieves 93.99% classification accuracy. While this reflects consistency with the derived supervision, we treat it primarily as validation of the labeling policy rather than a direct measure of real-world glare risk.**

***Index Terms*—adaptive headlights, glare avoidance, embedded vision, edge computing, real-time inference, lightweight CNN.**

I. Introduction

Cyclists ride across extreme lighting transitions, like direct sun, dusk, tunnels, unlit roads, and oncoming traffic. As bikes are intrinsically more unsafe than automobiles due to their size and road speed, manual headlight control does not scale to these transitions because the rider must keep attention on balance and situational awareness. Automotive adaptive headlight systems span sensor-first controllers and camera-based DNN pipelines with temporal post-processing [1]–[3]. While effective in vehicles with powerful compute and stable mounting, these architectures face three critical challenges on bike platforms: high computational cost from heavy models (e.g., object detectors), complex threshold tuning across modules, and sensitivity to vibration and rapid viewpoint changes inherent to cycling.

We address these challenges through a unified perception-to-control approach that consolidates illumination and hazard detection into a single lightweight residual CNN. Using a safety-first policy that prevents dazzling other riders with a high beam, our system ensures both road safety for the rider and other road users. By training end-to-end on derived labels and hazard bounding boxes from BDD100K [4], we eliminate hand-tuned thresholds and reduce compute by replacing heavy components, and finally consolidate the control policy into a single models that learns the decision boundary implied by a safety-oriented labeling rule.

Project conducted at the Technische Universität Nürnberg (UTN), in collaboration with Supernova GmbH.

II. Related Work

A. Specific AHB/ADB Studies

Sivapriya et al. propose a camera+GPS+microcontroller AHBM pipeline that uses grayscale conversion and color inversion to detect oncoming headlights while reducing confusion with street lamps [5]. Their validation is primarily MATLAB simulation and module-level prototype logic, and their conclusion reports better robustness than LDR-only approaches.

Ashqar et al. implement a low-cost embedded ML dipper using LDR features and a Decision Tree classifier [6]. They report 95% average accuracy, reduced false triggers versus fixed-threshold baselines, and response time below 200 ms, but perception is based on scalar illumination signals rather than scene structure.

Somasiri et al. evaluate detector families and deploy a YOLOv3-style regression detector for ADB operation [7]. Their reported average detection accuracy is 0.86 in nighttime and adverse-weather driving tests, with beam control handled after a separate detection stage.

Zakaria et al. combine YOLOv8 detection with camera-radar Kalman fusion, steering-angle sensing, and LDR input for adaptive LED and servo control [8]. They report 71.25% accuracy for YOLOv8-medium (5,671 images, 25 epochs), plus CARLA-based testing and hardware prototype validation.

Nkrumah et al. provide a review of 44 studies and group intelligent headlight control into sensor-based, ML-based, PWM-based, and fuzzy-logic-based families [1]. Their utilization analysis (2018–2022) reports 50% sensor-based, 45.5% ML-based, and 4.5% PWM/fuzzy approaches, and highlights ongoing ML sensitivity to nighttime/adverse-condition data quality.

B. Synthesis and Positioning

These studies show a consistent trade-off: simpler sensor-rule systems are efficient but perception-limited, while detector/fusion pipelines improve semantic awareness but increase calibration and integration overhead. Our single residual CNN targets this gap by predicting beam state directly from one camera stream, removing detector-to-controller handoff and reducing threshold tuning.

C. Supporting Context

Additional context comes from lightweight CNN design (MobileNet, EfficientNet) [9], [10], embedded deployment constraints on Raspberry Pi-class platforms [11], and single-network road-scene perception precedent from DeepTLR [12].

D. Comparison and Positioning

Table I compares these systems with our deployed model.

Compared with these pipelines, our implementation removes extra detector-to-controller bridges and sensor-fusion dependencies by learning beam actions directly from a single camera frame. This addresses three concrete limitations seen in prior work: threshold fragility between modules, calibration overhead from multi-sensor stacks, and poor scalability of manual event supervision. Our label construction follows weak-supervision principles, deriving beam actions from existing BDD100K annotations (time-of-day and hazard boxes) rather than frame-by-frame manual beam labels [4], [13].

III. System Architecture

A. Problem Formulation

We model headlight control as a three-class classification problem over forward-facing RGB images x:

$$f_\theta(x) \to y \in \{\texttt{off},\ \texttt{dim},\ \texttt{high}\} \tag{1}$$

where classes represent a conservative safety policy: `off` for bright daytime (no headlight needed), `dim` when dark with hazards present (avoid glare), and `high` when dark without hazards (maximize visibility).

B. System-Level Decision Rule and Temporal Stabilization

Although the perception model is trained as a three-class classifier, the deployed headlight controller operates at the *system level* by combining per-frame predictions with temporal stabilization to reduce flicker and prioritize safety-critical dimming.

Let x_t denote the RGB frame at time t. The CNN produces a probability vector over beam states:

$$p_t = f_\theta(x_t), \qquad p_t \in [0,1]^3, \quad \sum_{k=1}^{3} p_t^{(k)} = 1, \tag{2}$$

where the three entries correspond to $(\texttt{off},\ \texttt{dim},\ \texttt{high})$. We write

$$\hat{y}_t = \arg\max_k \ p_t^{(k)} \tag{3}$$

for the per-frame predicted class.

The deployed controller maps model outputs to an action $a_t \in \{\texttt{off}, \texttt{dim}, \texttt{high}\}$ using a temporal policy

$$a_t = \pi(s_{t-N+1:t};\ H,\ F,\ N,\ \tau)\,, \tag{4}$$

where $s_{t-N+1:t}$ denotes the recent history of dimming-evidence scores over the last N inference steps (equivalently $w = N/f_s$ seconds at inference rate f_s), and (H, F, N, τ) are stabilization hyperparameters described below.

Hazard evidence score. For the deployed single-classifier model (Variant E), we define the scalar dimming-evidence score directly from the three-class output as

$$s_t = p_t^{(\texttt{dim})}. \tag{5}$$

Because the training policy defines `dim` as "night + hazard present," a higher $p_t^{(\texttt{dim})}$ directly indicates stronger evidence for dimming. (For earlier ablation variants with an explicit hazard head, the analogous score is $p_t^{(\text{hazard})}$.)

Moving-average rule (fast dimming). Let $\bar{s}_t = \frac{1}{N}\sum_{i=0}^{N-1} s_{t-i}$. We trigger a dim request if

$$\text{DIM_REQ}(t) \iff \bar{s}_t \geq \tau. \tag{6}$$

This rule increases recall by aggregating weak per-frame evidence over time.

Hold and flicker control (cautious brightening). To prevent rapid oscillations, we introduce asymmetric stabilization:

- **Hold time H (seconds):** once the controller enters `dim`, it remains in `dim` for at least H seconds before allowing a transition to `high`.
- **Flicker window F (seconds):** after a beam transition, additional transitions are suppressed for F seconds unless a safety-critical dim request occurs.

In other words, dimming is allowed immediately when Eq. (6) fires, but brightening is delayed to avoid flicker.

Deployment settings are reported with the ablation best-settings table (Variant E) in Sec. V-E.

C. Design Evolution and Rationale (Overview)

Our perception stack evolved from modular detector-based pipelines to a single policy-trained classifier as we balanced robustness, supervision scale, and deployment complexity on Raspberry Pi hardware. This subsection only summarizes that trajectory; the formal Variant A–E definitions are given in Sec. IV-C. In short, we moved from ResNet18+YOLO baselines toward compact CNN alternatives and finally to BDD100K -based policy supervision with a single deployable model.

Although the modular relevance-CNN stage (formalized as Variant C in Sec. IV-C) achieves higher dimming recall on our on-bike benchmark, it relies on event-level manual labels (first-visible oncoming hazards) and a separate thresholded relevance head, which complicates maintenance and scaling. Our final deployed model (Variant E; Sec. IV-C) is trained on policy-derived supervision from BDD100K attributes and vehicle boxes, enabling large-scale training and improved cross-environment robustness while collapsing inference and control into a single deployable model file.

TABLE I: Comparison with recent AHB/ADB studies and our deployment.

Work	Perception Stack	Control Output	Reported Evidence
AHBM [5]	Camera + GPS + microcontroller with grayscale/inversion headlight extraction	High/low switching	MATLAB simulation and prototype logic; reports better vehicle-vs-streetlight handling than LDR-only baselines; no latency reported
ML Dipper [6]	LDR + Decision Tree on embedded controller	High/low switching	95% average accuracy, < 200 ms response, and reduced false triggers versus fixed thresholds
YOLO-ADB [7]	Camera-based YOLOv3 regression detection	Detection-driven ADB actuation	Average detection accuracy 0.86 in nighttime/adverse-weather drive tests
Fusion ADB [8]	YOLOv8 + camera/radar Kalman fusion + steering + LDR	Segmented LED dimming and servo steering	71.25% detector accuracy (YOLOv8-medium; 5,671 images; 25 epochs) with CARLA + hardware prototype validation
Review [1]	Comparative study of 44 authors	Taxonomy and adoption analysis	Sensor 50%, ML 45.5%, PWM/fuzzy 4.5%; notes ML sensitivity to image quality and nighttime data quality
Single CNN (ours)	**Single RGB camera + residual CNN trained on policy-derived labels**	**Direct 3-state beam action (`off`/`dim`/`high`)**	**93.99% policy-consistency accuracy on BDD100K test split and 20.4 ms inference on Raspberry Pi 5**

D. Control Logic for Modular Pipelines

For the modular variants, the controller applies a small set of conservative rules. At a high level, the system dims whenever the scene is dark and a hazard appears in a central region, and it keeps the light off in bright daytime scenes. This rule-based logic also defines the safety policy later used to derive supervision for the end-to-end model.

Input: frame x, illumination $\hat{t}(x)$, hazard score $\hat{h}(x)$
If $\hat{t}(x) = \texttt{day}$: output `off`
Else if $\hat{t}(x) = \texttt{night}$:
  If $\hat{h}(x) \geq \tau$: output `dim`
  Else: output `high`

We tune the hazard threshold τ and apply optional temporal stabilization to reduce flicker.

E. Hardware Pipeline and Design

Fig. 1 shows the wiring diagram used in this project. An ESP32-CAM streams MJPEG video over Wi-Fi to a Raspberry Pi 5 running inference. The CNN predicts beam states at 10Hz, and a temporal stabilization module applies hold times and hysteresis to reduce flicker. GPIO pins drive relay logic that switches headlight modes. All components must operate in outdoor rainy and snowy conditions while mounted on a bicycle. Therefore, we designed and 3D-printed custom enclosures to securely integrate the complete pipeline onto the bicycle. Fig. 3 shows the 3D case model used for mounting the full setup.

F. Label Derivation from BDD100K

Since BDD100K lacks explicit beam labels, we derive supervision using time-of-day attributes and hazard bounding boxes. Let $t(x)$ be the time attribute and $\mathcal{B}(x)$ the set of annotated boxes. We define hazard indicator $h(x)$:

$$h(x) = 1 \Leftrightarrow \exists b \in \mathcal{B}(x) : c(b) \in \mathcal{C}, a(b) \geq \tau, \hat{x}(b) \in [\alpha, \beta] \tag{7}$$

where $\mathcal{C} = \{\texttt{car}, \texttt{truck}, \texttt{bus}, \texttt{rider}, \texttt{person}\}$, $a(b)$ is box area fraction, $\hat{x}(b)$ is normalized horizontal center, and $(\tau, \alpha, \beta) = (0.004, 0.20, 0.80)$. The beam label is assigned inline as $y(x) = \texttt{off}$ if $t(x) = \texttt{daytime}$; $y(x) = \texttt{dim}$ if $t(x) = \texttt{night} \wedge h(x) = 1$; $y(x) = \texttt{high}$ if $t(x) = \texttt{night} \wedge h(x) = 0$.

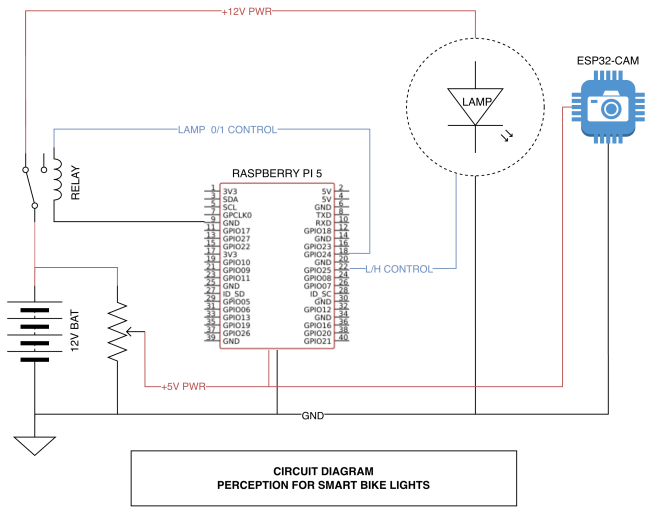

Fig. 1: Hardware wiring for ESP32-CAM streaming and Raspberry Pi GPIO control of a two-mode headlight (off/on and low/high).

This policy-based approach generates 70K+ training samples without manual annotation, enabling large-scale learning while maintaining interpretable supervision aligned with safety requirements.

G. Dataset Preparation and Analysis

We conducted extensive analysis of BDD100K label distributions to understand class imbalance and inform augmentation strategies. The derived training set contains 64,828 base samples (effective 76,006 after oversampling), with validation and test splits of 9,187 and 18,482 images respectively. Class distribution shows significant imbalance: `off` (56.5%), `dim` (34.9%), and `high` (8.6%).

The `high` underrepresentation stems from the conservative hazard definition—most night scenes contain at least one annotated vehicle within the central region. To address this, we apply targeted augmentation: gamma adjustments and noise injection for night scenes, and horizontal flips for daytime samples. Additionally, we repeat `high` indices with factor $K = 3$ during training to balance gradient contributions without disk-space overhead from duplicated images.

H. On-Bike Data Collection

To validate bike-specific behavior, we collected 1,979 real-world images (Day=743, Night=1,219) using the ESP32-CAM mounted on bike handlebars. This dataset captures unique challenges that are absent from automotive datasets: handlebar vibration, rapid pitch changes during braking and acceleration, lower mounting height (1.2m vs 1.5m for cars), and frequent transitions between tree shadows and direct sunlight on bike paths. We use these data for qualitative validation and ablation benchmarking rather than training, ensuring that the model generalizes from large-scale BDD100K data while remaining testable under bike-specific conditions.

High-rate event labeling requires synchronized, lossless video streams; however, ESP32-CAM MJPEG recording and transfer introduced dropped/corrupted frames under real riding conditions. Given this constraint, we prioritize approaches whose supervision scales from existing large datasets and does not depend on frame-exact event timing.

IV. Model Architecture and Training

A. Residual CNN Design

Our final architecture is a lightweight residual CNN with GroupNorm, squeeze-and-excitation blocks, and stochastic depth. The network follows a stem + four-stage pattern: a strided 7×7 convolution with max-pooling, followed by four residual stages that double channel width at each downsampling step. Base width is 48 channels with block depths $(2, 2, 2, 2)$.

GroupNorm stabilizes training independent of batch size, critical for edge deployment where batch size is constrained. Squeeze-and-excitation emphasizes informative channels under challenging night lighting. DropPath regularizes deep residual stacks by randomly dropping residual branches during training, improving generalization.

B. Training Configuration

We train on RGB images resized to 224×224 with ImageNet normalization. Augmentation includes color jitter (brightness/contrast/saturation/hue), random grayscale, and Gaussian blur. Class-conditional augmentation applies horizontal flips for `off` and additional gamma/noise perturbations for `high` to increase robustness under dark scenes.

The objective uses class-weighted cross entropy with label smoothing. We address class imbalance (8.6% `high` samples) by oversampling with repetition factor $K = 3$. Optimization uses AdamW with warmup+cosine scheduling. We maintain exponential moving average (EMA) of weights for evaluation stability and enable mixed precision training. Training uses seed 42 for reproducibility.

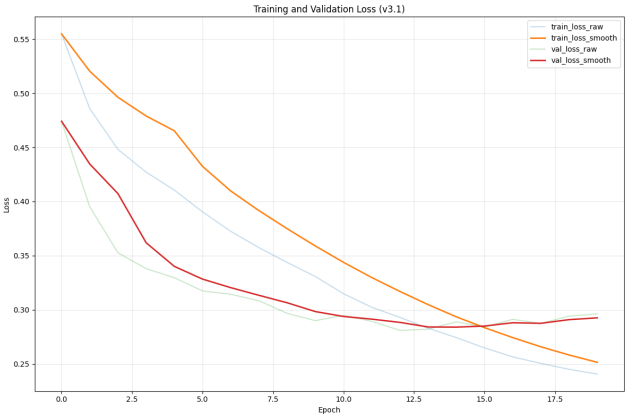

Fig. 2: Training and validation loss over epochs (v3.1).

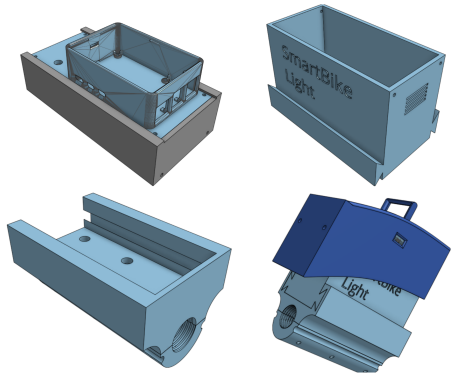

Fig. 3: 3D case model of the hardware enclosure used to mount the complete bike-light control setup.

Fig. 2 shows the training and validation loss of the final deployed single-CNN model (Variant E, v3.1). Training loss decreases steadily, while validation loss drops early and then plateaus with a slight late rise, indicating diminishing returns and mild overfitting after convergence.

C. Model Evolution and Variant Definitions

This subsection is the canonical definition of Variants A–E referenced in Sec. III-C. We iterate through five design variants, each addressing specific deployment challenges discovered during testing:

Variant A (ResNet18 + YOLO): Our initial baseline combined a ResNet18 illumination classifier with YOLOv8n for vehicle detection. While achieving strong accuracy, this approach suffered from three critical issues: (i) YOLO installation required complex dependency management on Raspberry Pi, (ii) combined inference time of 32.5ms limited control loop frequency, and (iii) threshold tuning between illumination confidence and detection scores proved fragile across different lighting conditions.

Variant B (Compact CNN + YOLO): We replaced ResNet18 with a lightweight CNN (3 convolutional layers,

TABLE II: Model variant evolution, supervision type, deployment complexity, and inference time.

Var.	Components	Supervision	Deploy.	Inf. (ms)
A	ResNet18 + YOLO	ImageNet + COCO	Multi	91.9
B	CNN + YOLO	ImageNet + COCO	Multi	70.8
C	CNN + CNN	Manual events	Multi	14.6
D	Multi-head CNN	Policy-derived	Single	11.1
E	Single CNN	Policy-derived	Single	20.4

1.2M parameters) for illumination classification, reducing that component's latency from 7.5ms to 3.0ms. However, YOLO remained the bottleneck at 25ms, and the modular architecture still required careful threshold coordination.

Variant C (CNN + CNN Relevance): This variant replaced YOLO with a compact "relevance CNN" that predicts binary hazard presence based on headlight-like patterns rather than explicit object detection. Training on manually labeled hazard/no-hazard samples from our bike dataset, this model learned to focus on high-contrast light sources. The combined pipeline achieved 14.6ms inference, but required maintaining two separate training datasets and introduced a new threshold between relevance score and beam decision.

Variant D (Multi-head CNN): We unified the backbone while maintaining separate output heads for environment (day/dawn/night) and hazard (present/absent). A shared feature extractor (base width 32, 3 stages) fed two classification heads. This design coupled feature learning while preserving interpretable intermediate outputs for debugging. Inference time dropped to 11.1ms, but the rule-based controller still required threshold tuning, and the multi-task loss weighting introduced additional hyperparameters.

Variant E (Single-head Residual CNN): The current system eliminates all intermediate outputs and trains directly on beam states. We increased model capacity (base width 48, 4 stages with residual connections) to compensate for the harder three-class problem. GroupNorm replaced BatchNorm for batch-size independence, squeeze-and-excitation blocks improved night-scene feature selection, and DropPath (rate 0.1) provided regularization. While inference time increased to 20.4ms compared to Variant D, this architecture requires zero threshold tuning, simplifies deployment to a single model file, and achieves better calibration on the BDD100K test split.

In this paper, *Variant E*, *Single CNN*, and *v3.1* refer to the same final deployed model unless explicitly stated otherwise.

The choice of Variant E over the faster Variant D reflects our prioritization of deployment simplicity and robustness over raw speed. Field testing revealed that threshold drift across different ESP32-CAM units and mounting positions caused inconsistent behavior in modular pipelines, while the end-to-end model maintained stable performance.

V. Experimental Results

Because this is a safety-critical controller, we treat BDD100K classification accuracy as a generalization sanity check, and we treat the on-bike benchmark as the deployment-validating evaluation.

TABLE III: Test metrics on BDD100K (18,482 images).

Class	Precision (%)	Recall (%)	F1 (%)
`off`	99.39	98.99	99.19
`dim`	94.40	88.58	91.39
`high`	64.99	83.03	72.91
Overall accuracy: 93.99%			

TABLE IV: On-bike benchmark (mean over 255 segments).

Model	Acc.	Prec.	Recall	FPR
ResNet+relevance (B)	0.773	0.773	0.976	0.471
CNN+relevance (C)	0.773	0.773	0.976	0.471
Multi-head (D)	0.695	0.788	0.658	0.150
v3.1 single (E)	0.730	0.796	0.808	0.306

A. BDD100K Test Split Evaluation

We evaluate the final deployed single-CNN model (Variant E, v3.1) on 18,482 BDD100K test images. It achieves 93.99% overall accuracy (Table III), with strongest performance on `off` and `dim`, while `high` remains harder due to class imbalance (8.6%) and ambiguity in "no hazard" scenes. Because labels are policy-derived, these metrics reflect policy consistency rather than direct glare safety, so on-bike results remain the primary deployment validation.

B. On-Bike Benchmark Evaluation

We evaluate on 255 labeled bike-ride segments using safety-oriented metrics. The benchmark treats dimming as the critical safety action and reports recall (dimming under hazards), false positive rate (unnecessary dimming), and latency. Composite score:

$$\text{score} = 0.55 \cdot \text{recall} - 0.25 \cdot \text{FPR} - 0.20 \cdot \text{latency}_{\text{norm}} \quad (8)$$

Table IV compares variants. The modular CNN+relevance pipeline (Variant C) achieves highest recall (97.6%) but higher FPR (47.1%). The single-head v3.1 model (Variant E) balances recall (80.8%) with lower FPR (30.6%), providing more stable behavior for real-world deployment.

C. Computational Performance

Table V shows inference time and model size. The single-head residual CNN achieves 20.4ms inference (49 FPS) on Raspberry Pi 5, enabling real-time operation with 10Hz control loop. This represents 37% speedup versus ResNet+YOLO baseline while maintaining comparable accuracy.

D. Qualitative Analysis

Qualitative inspection shows correct behavior across the three target conditions: bright daytime (`off`), night with oncoming vehicles (`dim`), and empty night roads (`high`). The main errors occur in twilight transitions, distant vehicles below the area threshold (< 0.004), and bright street-lit night scenes that resemble daytime; together, these indicate residual coupling issues between illumination and hazard cues. In on-bike tests, the model usually dims for oncoming cars at 30–50 m, but occasional misses appear for single-headlight motorcycles,

TABLE V: Computational performance on Raspberry Pi 5.

Model	Inference (ms)	Parameters (M)
ResNet18+YOLO	32.5	23.5
CNN+relevance	14.6	2.8
Single CNN (v3.1)	20.4	4.2

and parked cars with daytime running lights can trigger false dimming (mostly in parking lots).

E. Ablation Study

This section reports the ablation study used to select a deployment model and temporal control settings. The study prioritizess safety outcomes (dimming when hazards are present) while penalizing unnecessary dimming and slow response. The evaluation reuses caches raw model outputs to avoid repeated inference and ensures all models are compared under identical segments and trimming rules.

1) Protocol: Ground-truth labels are time-stamped beam states (`off`, `dim`, `high`) converted into intervals. Videos are segmented into fixed-length chunks of 30 seconds, and evaluated at three normalized resolutions (low, med, high). Include-range trimming is applied to exclude irrelevant portions of videos; segments are evaluated only if fully inside the include range. In this study, 255 segments were evaluated.

Temporal control is applied to each model using a grid over:

- hazard threshold τ
- averaging window in seconds N
- hold time in seconds H
- flicker window in seconds F

All metrics are computed on stabilized predictions (after averaging, thresholding, hold, and flicker control).

2) Metrics and Composite Score: We report recall, precision, false-positive (FPR), and latency. Latency is measured as the time from a group-truth transition to a stable predicted transition; when no match is found, a penalty of 10.0s is used. Model ranking uses a safety-first composite score:

$$\text{score} = 0.55 \cdot \text{recall} - 0.25 \cdot \text{fpr} - 0.20 \cdot \text{latency}_{norm}$$

where $\text{latency}_{norm} = \min(\frac{\text{latency}}{5.0}, 1)$

3) Results: The top models by composite score are shown in Table VI. While Variant C yields the highest dimming recall, it also produces substantially higher false dimming and depends on event-level hand labels that do not scale with additional environments or sensor configurations. Variant E is selected for deployment because it (i) removes inter-model thresholds, (ii) trains from scalable policy-derived labels, and (iii) reduces failure modes due to missing/corrupt frames during on-bike data acquisition.

TABLE VI: Top 3 Models (Mean over Segments)

Model	Score	Acc.	Prec.	Recall	FPR	Infer (ms)
C	0.219	0.773	0.773	0.976	0.471	14.6
B	0.219	0.773	0.773	0.976	0.471	32.5
E	0.185	0.753	0.793	0.861	0.354	20.4

4) Best Settings: Table VII lists the best temporal configurations per model from the tuning grid.

TABLE VII: Best Temporal Settings (per model)

Model	τ	N	H	F
A	0.40	10	0.5	3.0s
B	0.40	10	0.5	3.0s
C	0.30	10	0.5	2.0s
D	0.30	20	1.0	2.0s
E	0.30	20	1.0	2.0s

VI. Conclusion

We present a lightweight edge-based adaptive headlight system for bikes that consolidates perception and control into a single residual CNN. By training end-to-end on derived labels from BDD100K , our approach eliminates hand-tuned thresholds and achieves 93.99% accuracy while running at 49 FPS on Raspberry Pi 5. The system demonstrates practical deployment with ESP32-CAM streaming and GPIO actuation, providing a complete solution for bike safety applications.

On-bike evaluation reveals a system-level trade-off. In our experimental setup, the dual-model configuration—comprising separate illumination and hazard modules—achieves higher dimming recall when trained with event-level hazard-onset annotations. However, this configuration introduces additional supervision overhead and requires coordination between model outputs at runtime.

The proposed single-model architecture instead learns a direct mapping from image to beam state using policy-derived supervision from BDD100K. This reduces deployment complexity to a single inference graph and removes inter-model threshold tuning. In our deployment configuration, Variant E achieves 80.8% dimming recall with 30.6% false dimming on 255 segments at 20.4 ms inference time, providing a scalable and maintainable solution for embedded operation.

These results indicate that robustness and maintainability are driven more by scalable policy supervision and end-to-end decision learning than by maximizing isolated dimming recall in modular pipelines. They also reinforce that safety-oriented temporal stabilization is required for deployable on-bike behavior, since frame-level accuracy alone does not guarantee stable control.

Future work will address twilight transitions through targeted data collection, incorporate temporal modeling for improved hazard persistence detection, and extend the hazard proxy beyond geometric constraints to include distance and relative speed estimation. The complete system, including training code, deployment scripts, and evaluation benchmarks, is available for reproducibility.

Acknowledgment

The authors thank Supernova GmbH for hardware support and domain expertise.

REFERENCES

[1] J. K. Nkrumah, Y. Cai, and A. Jafaripournimchahi, "A review of automotive intelligent and adaptive headlight beams intensity control approaches," *Advances in Mechanical Engineering*, vol. 16, no. 4, p. 16878132231220355, Apr. 2024.

[2] N. Alarcon, "DNN-based perception for automatic high beam control," NVIDIA Technical Blog, Jan. 2020, accessed 2026-02-10. [Online]. Available: https://developer.nvidia.com/blog/dnn-based-perception-for-automatic-high-beam-control/

[3] NVIDIA, "Driveworks sdk reference: Lightsourcenet," NVIDIA DriveWorks SDK Documentation, 2020, accessed 2026-02-10. [Online]. Available: https://docs.nvidia.com/drive/driveworks-3.5/lightsourcenet_mainsection.html

[4] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, "Bdd100k: A diverse driving dataset for heterogeneous multitask learning," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[5] J. Sivapriya, M. Rp, S. B, and B. T. Srikanth, "Automatic headlight beam management system for vehicles," *International Journal of Innovative Technology and Exploring Engineering*, vol. 9, no. 7, p. 1089–1092, May 2020.

[6] A. A, J. N. M, and M. A. Alagar, "Automatic high beam dipper system using machine learning," *International Journal for Research in Applied Science and Engineering Technology*, vol. 13, no. 10, p. 1349–1357, Oct. 2025.

[7] N. P. Somasiri, S. Wickramasinghe, and S. Ganesan, "Deep learning model regression based object detection for adaptive driving beam headlights," in *Proceedings of the 2023 8th International Conference on Machine Learning Technologies*, ser. ICMLT '23. New York, NY, USA: Association for Computing Machinery, Jun. 2023, p. 235–239. [Online]. Available: https://doi.org/10.1145/3589883.3589919

[8] M. Zakaria, E. Yaser, M. Rooby, M. Hamed, M. Mahmoud, and A. M. Anis, "Adaptive headlight control and real-time pedestrian detection," *International Journal of Computer Applications*, vol. 186, no. 34, p. 18–25, Aug. 2024.

[9] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," 2017. [Online]. Available: https://arxiv.org/abs/1704.04861

[10] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," 2020. [Online]. Available: https://arxiv.org/abs/1905.11946

[11] A.-F. Khalifa, E. Badr, and N. Hesham, "A survey on human detection surveillance systems for raspberry pi," *Image and Vision Computing*, vol. 85, pp. 1–13, 03 2019.

[12] M. Weber, P. Wolf, and J. M. Zollner, "Deeptlr: A single deep convolutional network for detection and classification of traffic lights," in *Proceedings of the IEEE Intelligent Vehicles Symposium (IV)*, 2016, pp. 342–348.

[13] S. R. Cachay, B. Boecking, and A. Dubrawski, "End-to-end weak supervision," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021, 35th Conference on Neural Information Processing Systems. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/0e674a918ebca3f78bfe02e2f387689d-Paper.pdf